Zeluvi – Documento de System Design

Nov 9, 2026 · @Vinicius

Visão geral

O Zeluvi é uma plataforma web e mobile que centraliza comunicação, ocorrências, serviços e a **gestão financeira** de um condomínio para três perfis: morador, síndico/administração e portaria. Este documento descreve como o sistema será construído para suportar alto volume e exigências corporativas. Nesta versão, o módulo **Financeiro** passa a cobrir a Fase 2 (RF09 e RF10), com integração a um provedor de pagamento.

Objetivos técnicos

- Suportar 1 milhão de escritas para cada 2 milhões de leituras (detalhado no documento de Estratégia de Escalabilidade).
- Manter o backend stateless, para escalar com réplicas idênticas.
- Isolar os dados de cada condomínio (multi-tenant) e proteger dados pessoais conforme a LGPD.
- Oferecer interface acessível, inclusive para idosos, seguindo o guia de estilo do projeto.
- Processar pagamentos de forma idempotente e conciliada, sem guardar dados de cartão no Zeluvi.

Escopo

Dentro do escopo: módulos de Comunicação, Ocorrências, Serviços e **Financeiro**, os três perfis de usuário e os diferenciais com IA (Copiloto do Síndico, Motor de Automações, Painel Executivo, Morador 360° e Assembleias Digitais com transcrição por IA).

Fora do escopo: fabricação de hardware (o sistema só se integra por API a equipamentos existentes), contabilidade bancária profunda equivalente a um ERP e portaria remota sem supervisão humana. O Financeiro emite e acompanha cobranças e pagamentos; ele não substitui a contabilidade completa do condomínio.

Requisitos

A arquitetura é guiada por seis requisitos não funcionais, com metas propostas que a equipe deve validar. O Financeiro herda todas elas e adiciona exigências próprias de pagamento (última linha).

Requisito	Meta proposta	Como atendemos
Disponibilidade	99,9% ao mês	Réplicas da aplicação em duas zonas de disponibilidade e banco com failover automático
Latência	p95 abaixo de 300 ms na leitura e de 500 ms na escrita	Cache em Redis, índices no banco e trabalho pesado em fila
Escalabilidade	1 milhão de escritas para cada 2 milhões de leituras	Escala horizontal, réplicas de leitura e filas (ver Estratégia de Escalabilidade)
Segurança e LGPD	TLS 1.2 ou superior, criptografia em repouso e trilha de auditoria	Autenticação por token, controle de acesso por perfil e isolamento por condomínio
Recuperação	Perda máxima de 15 min e retorno em até 1 h	Backups contínuos com restauração para qualquer ponto no tempo
Acessibilidade	WCAG 2.2 nível AA	Componentes do guia de estilo, com textos grandes e contraste alto
Integridade financeira (novo)	Nenhuma cobrança duplicada e nenhum pagamento perdido	Chave de idempotência, webhooks assinados com reprocessamento e conciliação diária com o provedor

Requisitos funcionais por perfil

- **Morador:** relatar ocorrências, receber comunicados, fazer reservas, avisar mudanças e acompanhar solicitações. **Novo (RF09):** consultar boletos e a situação dos pagamentos, baixar a segunda via e pagar pelo app. **Novo (RF10):** consultar prestadores de serviço do condomínio e as perguntas frequentes.
- **Síndico / Administração:** gerenciar ocorrências e reservas, enviar comunicados, administrar cadastros de moradores e usar os recursos de IA (Copiloto, Automações, Painel Executivo, Morador 360° e Assembleias Digitais). **Novo (RF09):** emitir e cancelar boletos, acompanhar inadimplência e conciliar pagamentos. **Novo (RF10):** cadastrar prestadores e manter a base de perguntas frequentes.
- **Portaria:** registrar recebimento e entrega de encomendas, registrar avisos de mudança, acompanhar ocorrências relevantes e apoiar a comunicação do condomínio. **Novo (RF10):** consultar prestadores autorizados e as perguntas frequentes para

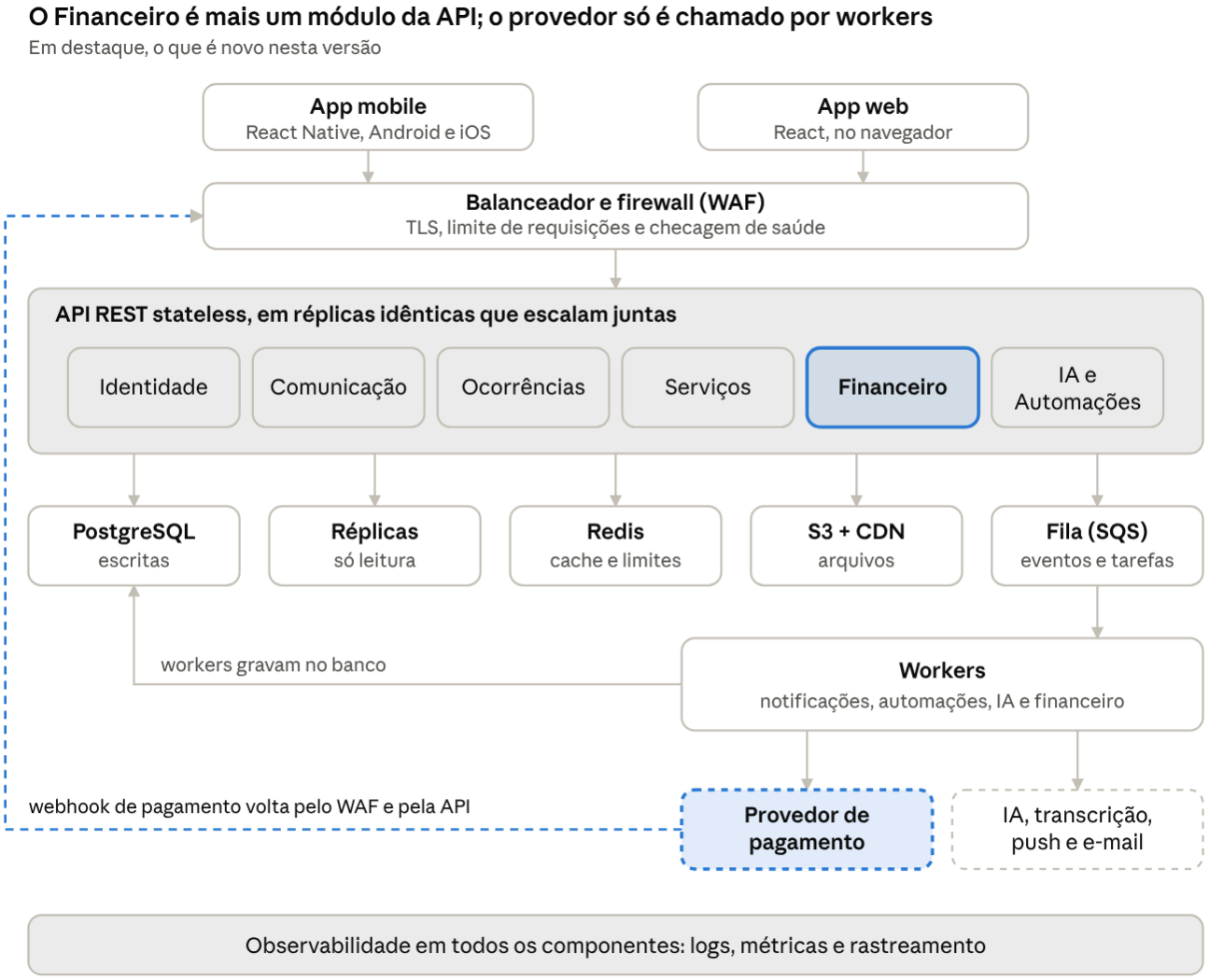
orientar visitantes e moradores. A portaria não acessa dados financeiros.

Rastreabilidade da Fase 2

Requisito	O que cobre	Módulo responsável
RF09	Boletos e situação de pagamentos	Financeiro
RF10	Prestadores e perguntas frequentes	Financeiro (submódulo Prestadores e FAQ)

Arquitetura de alto nível

A arquitetura separa o que o usuário espera ver na hora (a API) do que pode esperar alguns segundos (os workers, alimentados por uma fila). O Financeiro segue a mesma regra: a API grava a cobrança e publica um evento; só os workers falam com o provedor de pagamento.



arquitetura de alto nível · clientes, API com seis módulos, dados, fila, workers e provedores externos

Os apps chegam à API por um balanceador. A API grava no PostgreSQL, lê do cache e das réplicas e publica eventos na fila. Os workers cuidam de notificações, transcrições, automações e tarefas financeiras, e chamam provedores externos quando precisam. O aviso de pagamento confirmado faz o caminho inverso: o provedor chama o endpoint de webhook, que passa pelo mesmo balanceador e firewall da API.

Componentes e módulos

O backend é dividido em **seis módulos** (antes eram cinco) que compartilham o mesmo banco, mas só conversam entre si por interfaces internas e por eventos. Quando algo relevante acontece (uma ocorrência criada, uma reserva aprovada, um pagamento confirmado), o módulo publica um evento numa fila, e os módulos de Comunicação e de Automações reagem a ele sem travar a requisição de quem fez a ação.

Módulo	O que faz	Quem usa
Identidade e Acesso	Login, perfis e cadastro de moradores e unidades	Todos; o síndico administra os cadastros
Comunicação	Comunicados, notificações e mensagens diretas entre moradores e administração	Síndico envia; morador e portaria recebem
Ocorrências	Relato e acompanhamento de problemas: manutenção, limpeza, elevador, iluminação e outros	Morador abre; síndico gerencia; portaria acompanha as relevantes
Serviços	Reserva do salão e de áreas comuns, agendamento do elevador de serviço, encomendas e avisos de mudança	Morador reserva; síndico aprova; portaria registra encomendas e mudanças
Financeiro (novo)	Boletos e situação de pagamentos (RF09), integração com o provedor de pagamento, conciliação, cadastro de prestadores e perguntas frequentes (RF10)	Morador consulta e paga; síndico emite e concilia; portaria só consulta prestadores e FAQ
IA e Automações	Copiloto do Síndico, Motor de Automações, Painel Executivo, Morador 360° e Assembleias Digitais com transcrição	Síndico e administração

Eventos que o Financeiro publica e consome

- **Publica:** boleto emitido, boleto vencido, pagamento confirmado, pagamento falhou e inadimplência detectada. A Comunicação usa esses eventos para avisar o morador, e o Motor de Automações para regras de cobrança.
- **Consome:** unidade criada ou alterada e morador cadastrado (vindos de Identidade e Acesso), para gerar cobranças para a unidade certa.

Módulo Financeiro e integração de pagamento

O módulo Financeiro cobre a Fase 2: **RF09** (boletos e situação de pagamentos) e **RF10** (prestadores e perguntas frequentes). Os detalhes da API do provedor de pagamento e dos canais em tempo real ficam no documento **Mapeamento de Integrações**; aqui está só o desenho que o System Design precisa garantir.

Responsabilidades

- **Cobranças e boletos (RF09):** gerar a cobrança de cada unidade (taxa condominial e extras), emitir o boleto pelo provedor, oferecer a segunda via e acompanhar o status: emitido, pago, vencido ou cancelado.
- **Situação de pagamentos (RF09):** dar ao morador a visão do que está em aberto e do histórico, e ao síndico o painel de inadimplência por unidade e por período.
- **Prestadores e perguntas frequentes (RF10):** cadastro de prestadores de serviço do condomínio (nome, categoria, contato, situação) e base de perguntas frequentes mantida pela administração. Não há movimentação de dinheiro nesta parte.
- **Conciliação:** comparar, todo dia, o que o Zeluvi registrou com o que o provedor informa, e sinalizar diferenças ao síndico.

Fluxo de pagamento (RF09)

1. O síndico (ou uma regra do Motor de Automações) cria a cobrança de uma unidade. O Financeiro grava a cobrança no PostgreSQL com status `emitido` e publica um evento na fila.
2. Um worker chama o provedor de pagamento para emitir o boleto, usando uma chave de idempotência por cobrança, e guarda no banco só o identificador externo, a linha digitável e o link do boleto.
3. A Comunicação avisa o morador. O morador consulta e paga pelo app ou pelo banco dele.
4. O provedor envia um **webhook** de confirmação. A API valida a assinatura, grava o evento bruto e responde rápido; um worker atualiza o status para `pago` e publica o evento de pagamento confirmado.
5. A conciliação diária compara as cobranças com o extrato do provedor. Diferenças viram pendências para o síndico.

Integração com o provedor de pagamento

Ponto	Decisão de projeto
Chamadas ao provedor	Sempre por worker assíncrono, com nova tentativa e espera crescente; a tela do morador nunca espera o provedor
Idempotência	Uma chave por cobrança e por tentativa; repetir a chamada não gera segundo boleto

Ponto	Decisão de projeto
Webhooks	Assinatura verificada, gravados antes de processar, processamento idempotente e reprocessamento manual possível
Dados de cartão	Não passam pelo Zeluvi; o pagamento acontece na página do provedor ou em componente dele (escopo de conformidade PCI reduzido)
Segredos	Chaves de API e segredo de webhook no AWS Secrets Manager, uma credencial por ambiente
Provedor fora do ar	Cobranças já emitidas continuam consultáveis; novas emissões ficam na fila e saem quando o provedor voltar
Provedor	Em aberto: escolher o provedor de pagamento (ver Decisões em aberto)

Contrato com o Mapeamento de Integrações

O Mapeamento de Integrações detalha a API de pagamento e a comunicação por WebSocket. Ele deve definir, e este documento assume:

- os endpoints de emissão, consulta e cancelamento de boleto e o formato dos webhooks de pagamento;
- os códigos de erro e as regras de nova tentativa;
- o canal em tempo real que atualiza a tela do morador quando o pagamento é confirmado, sem precisar recarregar.

Qualquer mudança nesses pontos deve ser refletida nos dois documentos.

Modelo de dados

O banco é relacional (PostgreSQL) e gira em torno do condomínio: toda tabela tem um identificador do condomínio, um identificador único universal (UUID) e as datas de criação e alteração. A segurança em nível de linha do PostgreSQL impede que uma consulta enxergue dados de outro condomínio, mesmo que o código erre o filtro. As entidades financeiras seguem a mesma regra.

Entidade	Campos principais	Observação
Condomínio	nome, endereço, configurações	Raiz de todos os dados

Entidade	Campos principais	Observação
Unidade	bloco, número, condomínio	Liga moradores a um endereço
Usuário	nome, e-mail, perfil, unidade	Perfil: morador, síndico ou portaria
Comunicado e Leitura	título, texto, público, quem leu e quando	A confirmação de leitura gera uma escrita por morador
Ocorrência e Histórico	tipo, descrição, local, status, responsável	Cada mudança de status vira uma linha de histórico
Área comum e Reserva	área, data, horário, status, regras de uso	Restrição no banco impede reservas sobrepostas
Encomenda	unidade, recebida em, entregue em, quem retirou	Registrada pela portaria
Aviso de mudança	unidade, data, tipo (entrada ou saída)	Registrado pelo morador e visto pela portaria
Mensagem	conversa, remetente, texto	Canal direto entre morador e administração
Assembleia e Transcrição	pauta, data, texto transcrito, ata	A transcrição por IA é gerada por um worker
Regra de automação	gatilho, condição, ação, execuções	Usada pelo Motor de Automações
Evento de auditoria	quem, o quê, quando, de onde	Somente inserção; nunca é alterado
Cobrança (novo)	unidade, competência, valor, vencimento, status, chave de idempotência	Uma linha por unidade e período; status: emitido, pago, vencido ou cancelado
Boleto (novo)	cobrança, id externo no provedor, linha digitável, link, emitido em	Guarda só a referência; o documento fica no provedor
Pagamento (novo)	cobrança, valor pago, pago em, meio, id externo	Criado a partir do webhook confirmado; valores em centavos, nunca em decimal
Evento de webhook (novo)	provedor, id do evento, conteúdo bruto, processado em	Somente inserção; id do evento único evita processar duas vezes

Entidade	Campos principais	Observação
Conciliação (novo)	data, cobrança, situação (conferida ou divergente), observação	Gerada pelo worker diário
Prestador (novo)	nome, categoria, contato, situação (ativo ou inativo)	Cadastrado pela administração (RF10)
Pergunta frequente (novo)	pergunta, resposta, categoria, ordem, publicada	Mantida pela administração (RF10)

Arquivos (fotos de ocorrências, anexos de comunicados) ficam no armazenamento de objetos, e o banco guarda só o endereço e os metadados. Os boletos em PDF ficam no provedor; o Zeluvi guarda só o link.

Segurança, perfis e permissões

A segurança se apoia em três barreiras: quem é o usuário, o que o perfil dele pode fazer e de qual condomínio ele pode ver dados.

- **Perfis:** morador, síndico/administração e portaria. As permissões de cada perfil seguem a matriz de acesso do mapa de telas e são conferidas em toda rota da API, não só escondidas na tela. **Financeiro:** o morador vê apenas as cobranças da própria unidade; o síndico/administração emite, cancela e concilia; a portaria não acessa cobranças nem pagamentos, só prestadores e perguntas frequentes.
- **Autenticação:** senha com hash Argon2, token de acesso de curta duração e refresh token rotativo. Proposta: verificação em duas etapas obrigatória para síndico e administração, que podem aprovar, apagar, enviar comunicados e emitir ou cancelar cobranças.
- **Isolamento entre condomínios:** filtro no código e segurança em nível de linha no banco, como segunda trava.
- **Proteção dos dados:** TLS 1.2 ou superior em todo tráfego, criptografia em repouso no banco e nos arquivos, e segredos guardados em um cofre (AWS Secrets Manager), nunca no código.
- **Pagamentos:** o Zeluvi não armazena número de cartão nem dados de autenticação; só referências do provedor. Webhooks são aceitos apenas com assinatura válida, e as chaves do provedor têm permissão mínima.
- **LGPD:** coletar só o necessário, registrar o consentimento, permitir que o titular consulte, corrija e apague seus dados e registrar cada acesso a dados pessoais. Assembleias transcritas por IA avisam os participantes antes de gravar. A situação de

pagamento é dado pessoal: um morador não vê a inadimplência de outro, e relatórios de inadimplência são restritos à administração. Registros fiscais e financeiros podem exigir retenção por prazo legal, que deve ser definido (ver Decisões em aberto).

- **Auditoria:** ações sensíveis (aprovar reserva, encerrar ocorrência, alterar cadastro, **emitir, cancelar ou baixar uma cobrança, e aprovar uma conciliação**) geram um evento que só admite inserção.
- **Proteção de borda:** firewall de aplicação (WAF), limite de requisições, validação de toda entrada e verificação automática de dependências vulneráveis a cada entrega. O endpoint de webhook tem limite próprio.
- **Hardware existente:** câmeras e catracas, quando integradas, usam chaves de API próprias e com permissão mínima.

Infraestrutura, implantação e observabilidade

O sistema roda na AWS, na região de São Paulo, o que mantém os dados no Brasil (importante para a LGPD) e reduz a latência para os moradores.

Infraestrutura

- Rede privada (VPC): só o balanceador fica em sub-rede pública; aplicação, banco e cache ficam em sub-redes privadas.
- Aplicação em contêineres (ECS Fargate), com réplicas em duas zonas de disponibilidade.
- Toda a infraestrutura descrita como código (Terraform), para recriar qualquer ambiente do zero.
- **Saída para o provedor de pagamento:** os workers chamam o provedor por saída controlada (NAT) para fora da VPC; o endpoint de webhook entra pelo balanceador e pelo WAF, como o resto da API.
- **Fila dedicada:** as tarefas financeiras (emissão de boleto, processamento de webhook, conciliação) usam uma fila própria no SQS, com fila de mensagens mortas, para que uma falha do provedor não atrase notificações nem automações.

Implantação

1. A cada alteração, o GitHub Actions roda os testes e o build.
2. A versão vai para homologação e passa por testes automáticos. O Financeiro usa o ambiente de testes (sandbox) do provedor, nunca credenciais de produção.
3. Após aprovação, a produção recebe a versão de forma gradual, trocando as réplicas aos poucos. Se os alertas dispararem, a versão anterior volta automaticamente.

Observabilidade

- Logs estruturados, com um identificador que acompanha a requisição do app até o banco. Logs financeiros nunca registram dados pessoais além do necessário.
- Métricas principais: latência p95, taxa de erro, tamanho da fila, CPU e conexões do banco.
- **Métricas do Financeiro:** taxa de sucesso na emissão de boletos, atraso entre pagamento e webhook, tamanho da fila financeira e da fila de mensagens mortas, e número de divergências na conciliação.
- Rastreamento distribuído (OpenTelemetry) e alertas para a equipe quando uma meta de latência ou de disponibilidade é violada, ou quando webhooks deixam de chegar por um período anormal.

Riscos e decisões em aberto

O maior risco é dimensionar errado por não saber em que período a meta de 1 milhão de escritas e 2 milhões de leituras se aplica. Por isso o sistema é projetado para o cenário mais pesado (por hora). Os demais riscos estão abaixo.

Risco	Impacto	Resposta
Custo das chamadas de IA	Conta mensal alta por condomínio	Limite de uso por condomínio, processamento assíncrono e cache de respostas repetidas
Provedor de IA fora do ar	Copiloto e transcrição indisponíveis	Falha isolada: comunicados, ocorrências e reservas funcionam sem a IA
Gravação e transcrição de assembleias (LGPD)	Risco jurídico	Aviso e consentimento antes de gravar, e prazo de retenção definido
Sinal fraco no celular	Morador perde uma ação	Cache de leitura e repetição segura de escritas com chave de idempotência
Equipe pequena para operar o sistema	Falhas sem resposta rápida	Serviços gerenciados, infraestrutura como código e alertas

Risco	Impacto	Resposta
Provedor de pagamento fora do ar (novo)	Novos boletos não saem; pagamentos demoram a aparecer	Fila financeira com nova tentativa; cobranças já emitidas continuam consultáveis; resto do sistema segue funcionando
Webhook perdido ou duplicado (novo)	Pagamento não baixado ou baixado duas vezes	Id de evento único, processamento idempotente, conciliação diária e reprocessamento manual
Cobrança errada ou duplicada (novo)	Perda de confiança e disputa com o morador	Chave de idempotência por cobrança, auditoria de emissão e cancelamento, e revisão do síndico antes de emissões em lote
Exposição de dados de inadimplência (novo)	Risco jurídico e constrangimento entre vizinhos	Acesso por perfil e por unidade, isolamento por condomínio e registro de cada consulta

Decisões em aberto

- Confirmar o período da meta de carga (por dia ou por hora).
- Validar a nuvem (AWS foi a proposta) e escolher os provedores de IA e de notificação push.
- Definir o prazo de retenção de dados pessoais, transcrições e auditoria, incluindo os registros financeiros.
- **Novo:** escolher o provedor de pagamento (boleto, e se haverá Pix e cartão além do boleto) e confirmar quem arca com as tarifas: o condomínio ou o morador.
- **Novo:** definir as regras de cobrança (multa, juros, desconto por pagamento antecipado) e se o Zeluvi calcula ou apenas repassa o que o condomínio configura.
- **Novo:** confirmar com o jurídico se o Zeluvi atua só como tecnologia ou como intermediador de pagamentos, o que muda as obrigações regulatórias.
- **Novo:** decidir se o cadastro de prestadores e a base de perguntas frequentes ficam sob revisão do síndico antes de publicar.

Próximo documento: o **Mapeamento de Integrações** (API de pagamento e comunicação por WebSocket) complementa este desenho e detalha o que a seção do Financeiro assume.